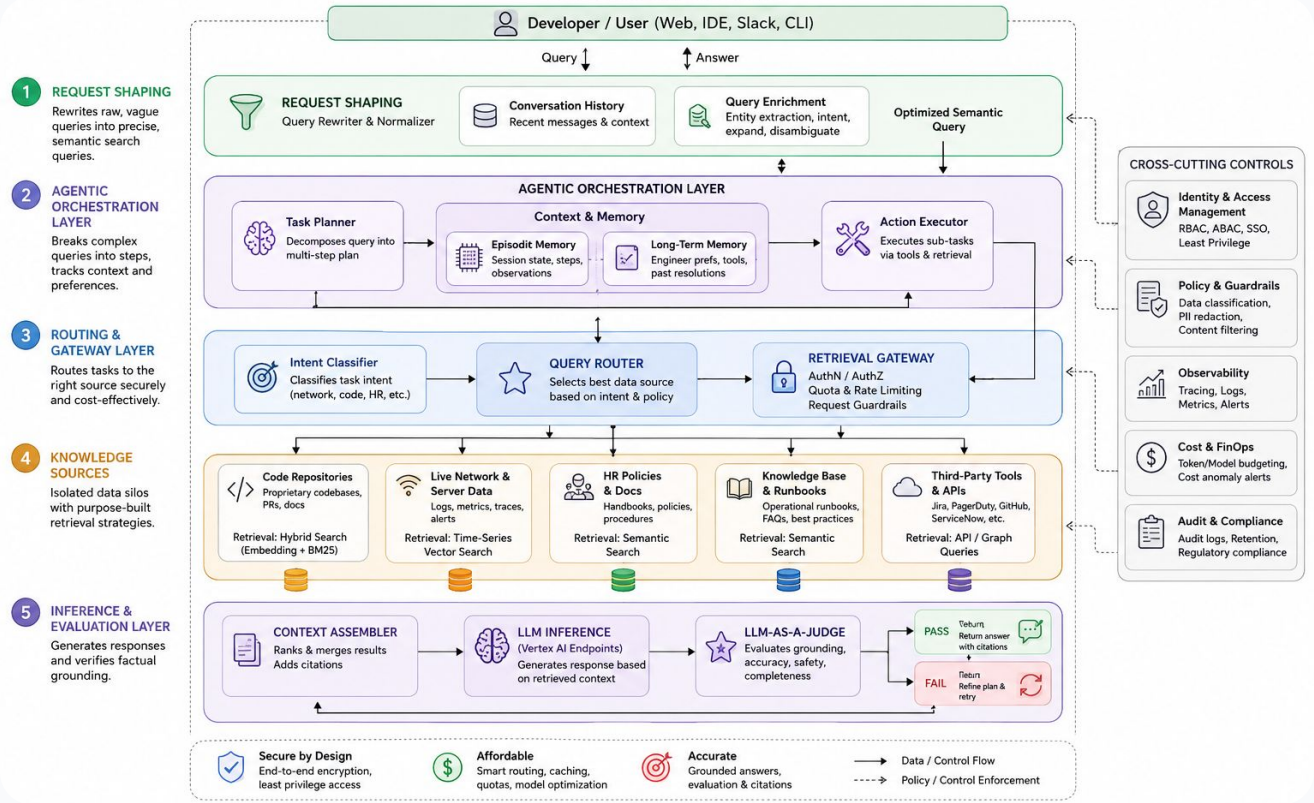
Retrieval Augmented Generation (RAG)
Module 2 Lecture 6

Pawan Goyal
Department of Computer Science and Engineering
IIT Kharagpur





Project Recap: Enterprise DevOps & Support Agent



Recap: The Limits of Linear RAG

Standard Linear Pipeline

Embed Query → Retrieve Top-K → Generate Answer

Assumption 1
Perfect Retrieval

Assumption 2
Complete Context

Critical Note: Linear pipelines cannot reflect, pause, or loop back when encountering multi-hop queries.

The "So What?": Passive to Active Solving

Iterative Reality
Resolving incidents in software is an iterative loop.

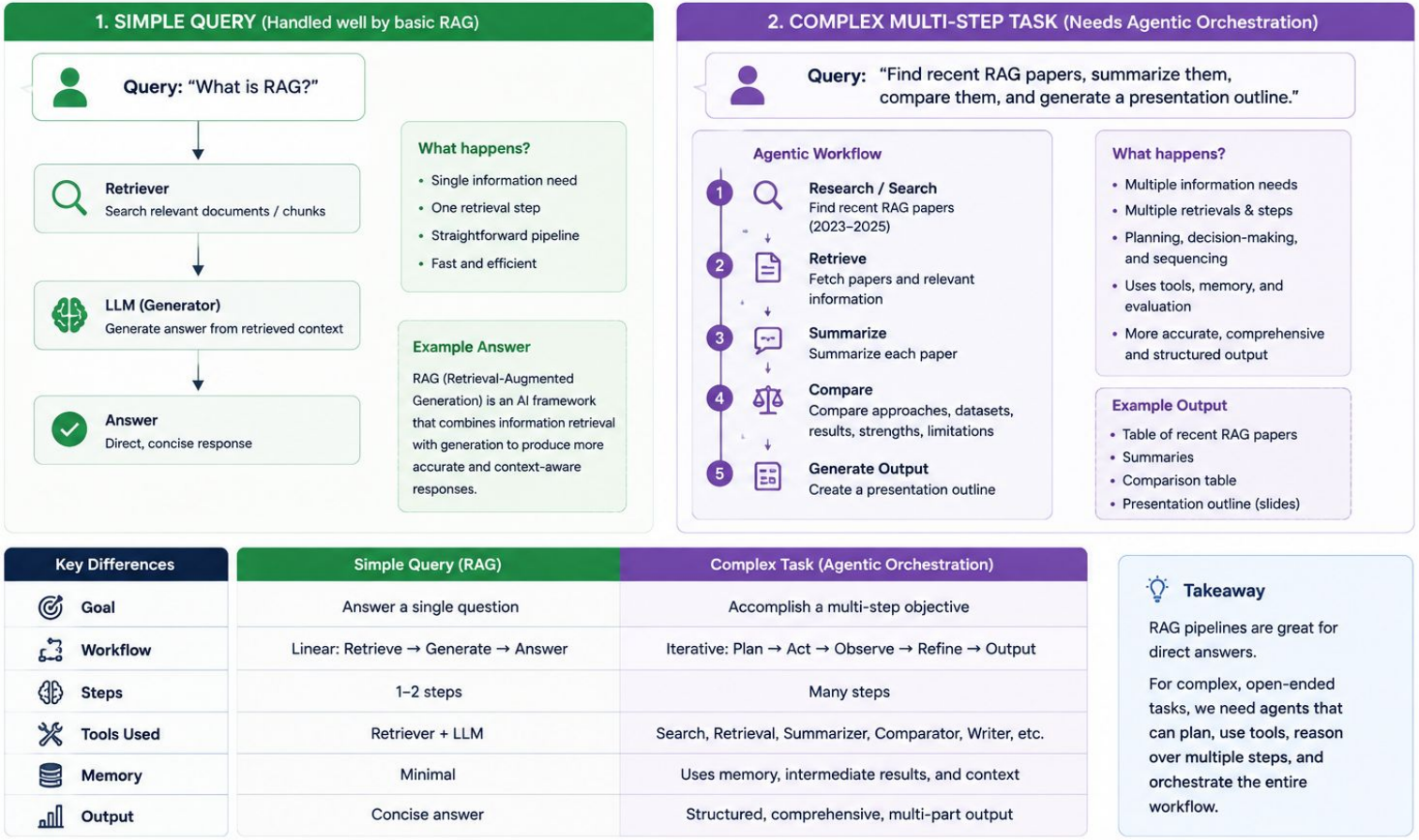
Responsibility Shift
Move from static code to handing control to the LLM.

The Core Objective

Shift from "Passive Reading" to "Autonomous Action"

The Essence of Agentic AI: A system that autonomously decides how to use tools, rather than just reading data.

Simple vs Complex Tasks



Key Differences	Simple Query (RAG)	Complex Task (Agentic Orchestration)
Goal	Answer a single question	Accomplish a multi-step objective
Workflow	Linear: Retrieve → Generate → Answer	Iterative: Plan → Act → Observe → Refine → Output
Steps	1–2 steps	Many steps
Tools Used	Retriever + LLM	Search, Retrieval, Summarizer, Comparator, Writer, etc.
Memory	Minimal	Uses memory, intermediate results, and context
Output	Concise answer	Structured, comprehensive, multi-part output

Takeaway

RAG pipelines are great for direct answers.

For complex, open-ended tasks, we need agents that can plan, use tools, reason over multiple steps, and orchestrate the entire workflow.

Bridging the Gap: Pipelines to Agents

The Evolution of Query Routing

Last Lecture: Passive routing via logical rules, embeddings, or simple LLM classifiers.

Today: **Cognitive Orchestration**. A central **Triage Agent** dynamically delegates tasks to Specialist Agents.

The Evolution of Request Shaping

Last Lecture: Explicit rewriting of vague queries into structured search strings.

Today: **Implicit Planning**. The plan itself becomes the perfectly shaped directive for downstream execution.

The Paradigm Shift

Upgrading from **passive pipelines** (static if/else) to an **active, autonomous team** negotiating complex solutions.

Agentic RAG: From Pipeline to Control Loop

Linear RAG (Last Lecture) retrieve once, then generate. Query shaping and routing were **static pre-processing** steps applied before a single retrieval call.

Agentic RAG (Today) retrieval becomes an **action the LLM chooses**. The agent decides whether to retrieve, what to retrieve, when to re-retrieve, and when it has enough evidence to answer.

Routing & shaping become dynamic the same decisions you hard-coded last lecture are now made **inside the loop**, conditioned on observations retrieved so far.

Key framing: Agentic RAG = RAG where retrieval, routing, and query rewriting are tool calls inside an Observe-Think-Act loop, not fixed stages in a DAG.

Basics of Agentic AI

The Foundation

LLM

Next-token predictor

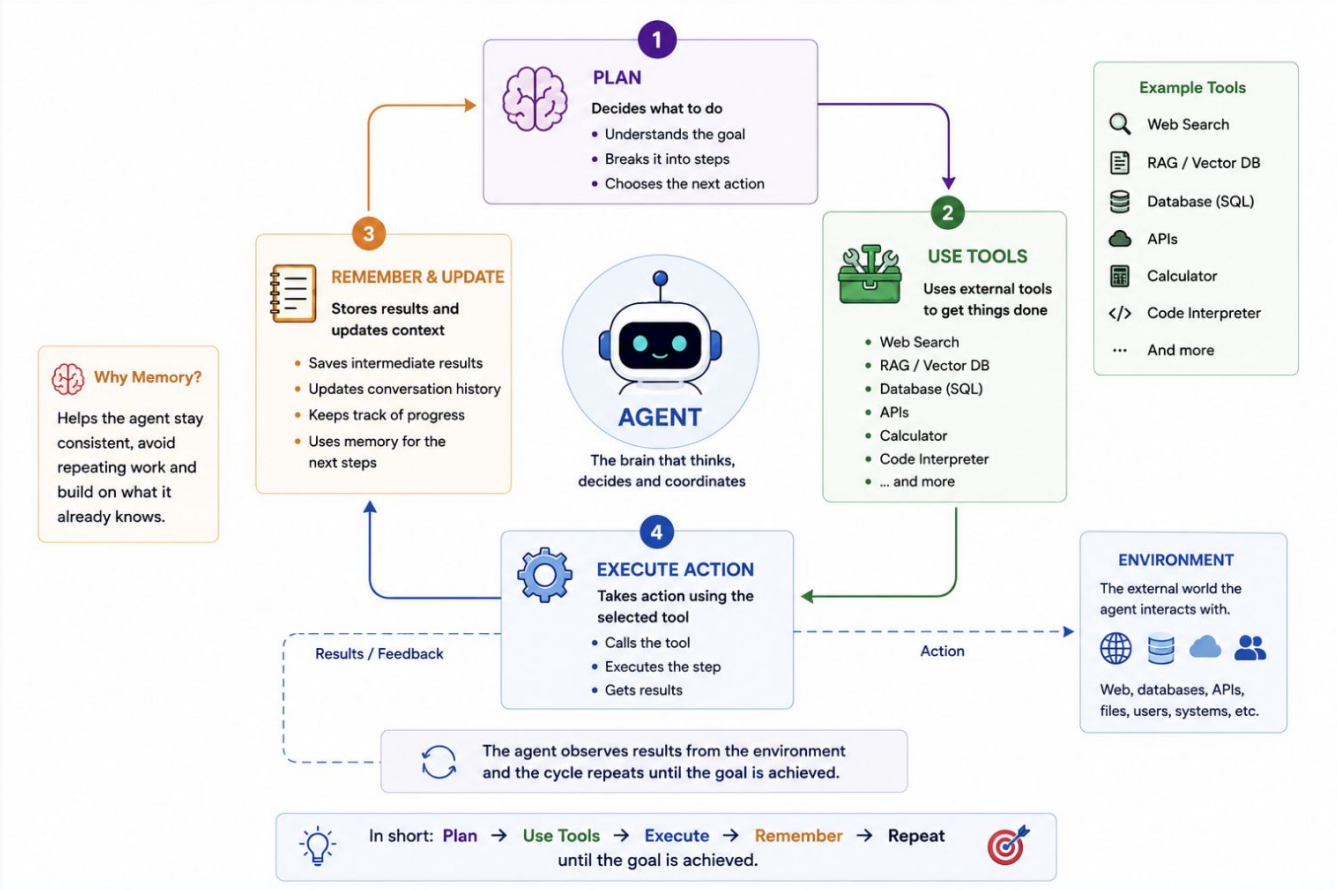
The Evolution

Agent

An LLM equipped with Tools, Memory, and a Planning mechanism

Key Architectural Insight: An agent is an architectural wrapper around an LLM that gives it agency—the ability to act on its environment.

Anatomy of an agent



The Core Loop of an Agent



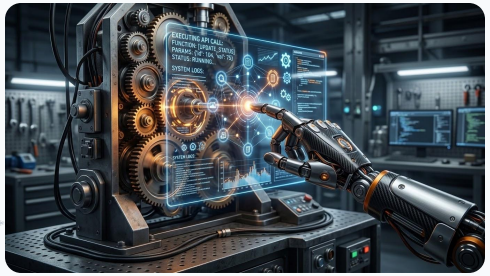
Observe

Receive user input or system state.



Think

The LLM decides what to do based on prompts and tools.



Act

Execute a tool (e.g., query a DB, call an API).

Note: This is often called the ReAct (Reasoning and Acting) framework. It mimics how humans troubleshoot issues.

Example of Agentic AI in Project Context

Primary Goal

Autonomously debug failing pipelines

TOOLS AVAILABLE

- **Vector DB:** Runbooks
- **Splunk API:** Logs
- **GitHub API:** Commits

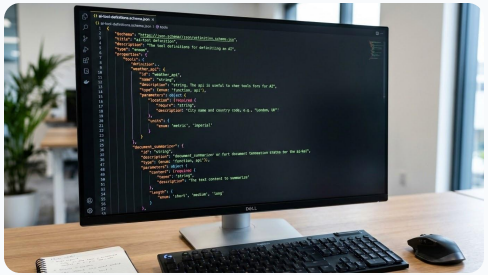
CAPABILITY EVOLUTION

PREVIOUSLY
Just fetched runbooks

NOW (FULL AGENTIC LOOP)
Fetch runbook → Read log → Check history → Write summary

Note: The system transition from basic retrieval to a multi-step analytical reasoning loop.

Tool Calling: The Engine of Agency



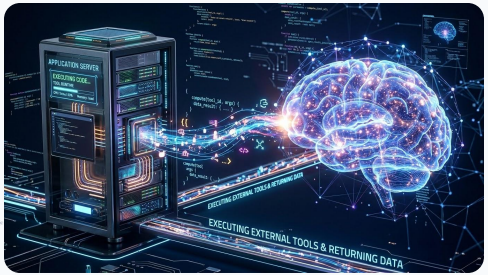
1. Tool Definition

Tools passed as JSON schemas describing functions and parameters.



2. LLM Response

The LLM requests a function call instead of generating raw text.



3. Execution

The app logic executes the function and returns results to the LLM.

Note: The LLM doesn't run code directly. It generates parameters for the application to process and return.

Designing a Tool for the DevOps Agent

Tool Definition

`fetch_server_logs`

Retrieve recent logs to diagnose failures, errors, or performance issues.



Why Descriptions Matter

LLMs choose tools based on **descriptions**, not just names.

✅ Helps agent know **when** to use the tool
✅ Helps agent understand **returned data**
❌ Avoids incorrect or hallucinated calls

Troubleshooting Example

Request: "Investigate 500 errors on db-12"

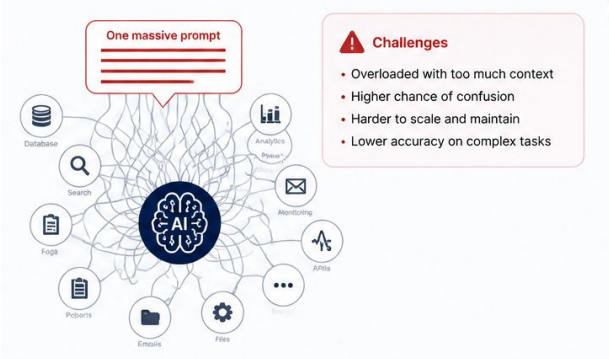
```
fetch_server_logs(
  server_id="db-12",
  minutes_ago=10
)
```

Result:
"Database connection pool exhausted"

Conclusion:
Connectivity issue, not code failure.

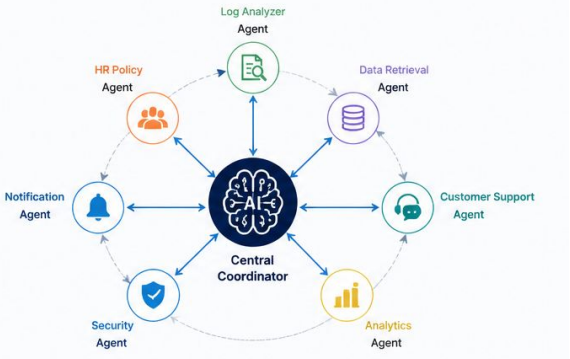
Key Lesson: The more clearly you describe a tool's purpose, the better the LLM can decide when to use it.

Theory: The Multi-Agent Concept



Single Agent

One massive prompt, all tools. Prone to confusion.



Multi-Agent

Specialized agents (e.g., Log Analyzer, HR Policy) working together.

Note: As systems scale, giving 50 tools to one LLM causes context exhaustion. We split them up.

Project Context: The DevOps Multi-Agent Team

To avoid context exhaustion, we divide our Enterprise DevOps Agent into a team of specialized sub-agents:



The Triage Agent

Acts as the front door. Analyzes user queries and delegates tasks to specialists.



Infrastructure Agent

Specializes in live telemetry. Queries Splunk, Datadog, and K8s clusters.



Codebase Agent

Specializes in source code. Searches GitHub, reads PR diffs and commits.



Policy & HR Agent

Specializes in RAG retrieval over runbooks and company HR policies.

Note: This is the cast of characters for our project. Instead of giving 50 tools to one LLM and risking hallucination, we give 3-5 highly relevant tools to each specialist.

As we dive into LangGraph later, we will learn how to wire these agents together so they can pass information back and forth to solve complex, multi-hop incidents.

Transitioning to Orchestration

The Core Challenge

How do we manage multiple tools and steps without getting stuck in infinite loops?

The Solution: Orchestration

Coordinating autonomous actions into a cohesive workflow to ensure reliability and termination.

Technical Architecture: Planners, Executors, and State Machines form the foundation of modern AI agent orchestration.

Agentic Orchestration



The Planner

The "Thinking" Phase



The Executor

The "Doing" Phase

Core Principle: To build reliable agents, we separate the "thinking" (Planning) from the "doing" (Execution) to ensure logical consistency and error handling.

What is a Planner?

Role

Breaks down a complex user request into a step-by-step execution plan.

Mechanism

Uses an advanced reasoning model (e.g., GPT-4) to generate a DAG (Directed Acyclic Graph) of tasks.

Key Constraint

The planner doesn't run tools. It just outputs a recipe.

Example: Step 1: Check logs. Step 2: Query codebase.

Note: Planners focus on the logic and structure of the solution, delegating the actual tool use to Executors.

What is an Executor?

Role

Takes a single step from the Planner and executes it.

Mechanism

Uses smaller, faster models to format tool calls and handle the exact API interactions.

Key Focus

The executor focuses narrowly on safety and delivery.

It prioritizes getting the job done safely and returning the payload to the orchestrator.

Note: Executors are the operational layer, translating logical steps into concrete, safe system interactions.

Why separate Planning and Execution?

Accuracy

Reduces cognitive load on the LLM.

Cost Efficiency

Use expensive models for planning and cheaper models for routine executions.

Fault Tolerance

If Step 2 fails, we don't need to re-plan Step 1.

Note: This is a crucial production design pattern.

Case Study: The Planner in Action

User Query:
"Payment gateway failing after last night's update."

Triage Agent (The Planner) Strategy

01. Code Analysis

Delegate to the **Codebase Agent** to fetch the latest commit diff from the `payment-gateway` repository.

02. Log Retrieval

Delegate to the **Infrastructure Agent** to pull error logs for `pay-pod-1` covering the last 12 hours.

03. Synthesis

Synthesize findings to compare code changes against live error traces for a final root-cause answer.

Under the Hood: The Triage Agent acts as an orchestrator. It does not run tools directly; instead, it formulates the recipe and routes steps to domain experts.

The Specialist Agents in Action (Execution Phase)

Execution Step 1 (Codebase Agent)

Receives directive from Planner. Formats GitHub API tool call to fetch recent code modifications:

```
github.get_commit_diff(repo="payment-gateway", sha="latest")
```

Observation: Returns Commit Hash **`a1b2c3d`** (Notes: "Added new strict rate-limiting logic").

Execution Step 2 (Infrastructure Agent)

Receives next directive. Formats Splunk API tool call to inspect live container error outputs:

```
splunk.query_logs(service="pay-pod-1", level="ERROR")
```

Observation: Returns **`TimeoutException at Auth.py line 42`** for `pay-pod-1`.

The specialist agents operate in a tight loop to accomplish narrow tasks safely. By splitting the work, we avoid overwhelming a single agent, reducing hallucinated API calls.

Note: Specialist agents act as the executors, returning specific payloads to the orchestrator.

Summary: Planners v/s Executors

Aspect	Planner	Executor
Primary Role	Decides what should be done	Carries out the decided actions
Main Question	"What is the best way to solve this task?"	"How do I perform this step?"
Input	User goal/task	Plan or action generated by planner
Output	Sequence of steps (plan)	Tool calls, actions, results
Focus	Reasoning and task decomposition	Execution and interaction with tools
Uses Tools?	Usually selects tools	Actually invokes tools
Example Action	"Search papers → Summarize → Compare"	Execute search, retrieve papers, generate summaries
Failure Mode	Bad plan, missing steps	Wrong tool call, execution error
Human Analogy	Project Manager	Engineer / Worker
In Project	Triage Agent	Tools, Codebase Agent, HR & Policy Agent

The Re-Planning Phase

Key Question:
What happens when an Executor fails or finds new information?

01. Feedback Loop

The Orchestrator feeds the result back to the Planner.

02. Dynamic Update

The Planner updates the plan dynamically based on new constraints.

Example: Network Failure

If the server is unreachable, the plan must pivot to checking the network status before retrying API calls.

Note: Dynamic environments require dynamic plans. Continuous re-planning ensures agent robustness.

Planning: Single-Agent ReAct

Seminal Work: "ReAct: Synergizing Reasoning and Acting in Language Models" (Yao et al., ICLR 2023)

The Core Thesis

Human intelligence seamlessly integrates task-oriented actions with verbal reasoning. ReAct enables LLMs to interleave generating *reasoning traces* (Thoughts) and *task-specific actions* (Tool use).

Key Synergy:

- Reasoning guides action
- Action grounds reasoning in external reality

Note: Before ReAct, reasoning (CoT) and acting (Tool use) were treated as separate domains.

The Limitation of Pure Reasoning (CoT)

Chain-of-Thought (CoT) allows models to solve complex problems by outputting intermediate steps.

The "Hallucination Cliff"

In closed-book CoT, if the model makes a factual error in Step 2, the error compounds uncontrollably through Step 10.

The ReAct Solution

ReAct forces the agent to pause reasoning, take an Action, and wait for an Observation (ground truth) before thinking again.

ReAct solves this by fact-checking itself against reality via tools.

Trivia: "Have you ever confidently reasoned your way to the wrong answer because your initial assumption was flawed?"

The Formal Definition of the ReAct Loop

Agent Input: The agent receives a user instruction **X**.

Agent Context (c_t)
The history of previous steps: $c_t = (x, t_1, a_1, o_1, \ldots, t_{t-1}, a_{t-1}, o_{t-1})$

Next Step Generation ($\hat{y}$)

Thought $t_t \in \mathbf{T}$: Internal state update to prepare for better actions.

Action $a_t \in \mathbf{A}$: External environment interaction.

Observation o_t: Environment returns ground truth to ground reasoning.

Note: Thoughts and Actions share the same context window. Thoughts prepare the model; actions inject empirical facts (o_t) back into the context.

Agent Context Example

The DevOps Translation (What this looks like in our code):

x (User Input): *"The auth-service is failing."*

t_1 (Thought): *"I need to check the error logs for auth-service to see why it's failing."*
a_1 (Action): `fetch_server_logs(service="auth-service", timeframe="1h")`
o_1 (Observation): `[API Return] "ConnectionTimeout: Cannot reach db-primary"`

t_2 (Thought): *"The issue isn't the auth-service code, it's the database connection. I need to check db-primary status."*
a_2 (Action): `query_infrastructure(target="db-primary")`

Scaling ReAct: The Context & Tool Exhaustion Problem

The Single-Agent Bottleneck

As enterprise systems grow, the Action space (**A**) expands to dozens of tools (Splunk, GitHub, Datadog, Jira).

If $|\mathbf{A}| > 20$, standard ReAct agents suffer from **"Tool Hallucination"** - they start guessing API parameters or mixing up tool schemas.

The Solution: Hierarchical Planning

Multi-Agent Systems enable distributed focus.

In enterprise DevOps, the environment is too vast for one ReAct loop. This is why we introduced our **Triage, Infra, and Codebase agents**.

Note: The ReAct paper tested models on simple environments (like Wikipedia). Enterprise scale requires specialized hierarchical structures.

Hierarchical ReAct: Triage as a Meta-Planner

The Meta-ReAct Loop: The Triage Agent orchestrates high-level strategy rather than executing API calls.

Shift in Action Space (A)

Action space consists of **Delegations to Specialist Agents** instead of raw API tools.

Execution Example

Thought: **"I need to know what code changed yesterday."**

Action: `DelegateTo(CodebaseAgent, "Fetch yesterday's commits")`

Observation: **Synthesized summary of commits returned by specialist.**

Note: The Triage Agent never touches raw JSON or API keys. It reasons purely at the strategic level, treating specialist sub-agents as its "Tools."

Formalizing the Multi-Agent State Transition

Agent Identity Tracking

We expand the ReAct context vector to track provenance:

$$c_{global} = (x,\ t_1^{Triage},\ a_1^{Triage \to InfrA},\ o_1^{Infra},\ t_2^{Triage},\ a_2^{Triage \to Codebase},\ \ldots)$$

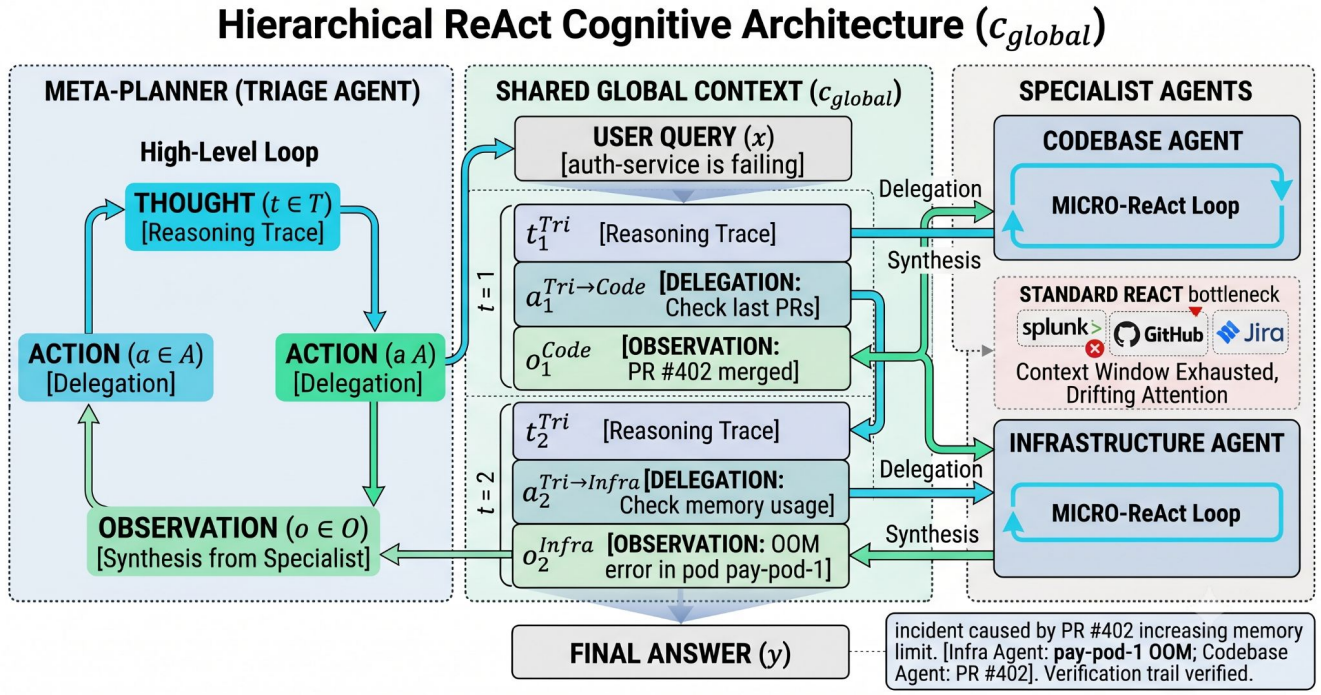
Isolated Infra Execution

The **Infra Agent** runs its own isolated micro-ReAct loop to query Splunk.

It returns only the final o^{Infra} back to the global state, shielding the Triage agent from raw system logs.

LangGraph Mechanism (Will come up in future slides): This formalizes LangGraph's "under the hood" operations. It manages the global vector and ensures the Triage agent receives clean, synthesized observations from specialists.

Hierarchical ReAct Cognitive Architecture (c_{global})



Key Idea: The Meta-Planner decomposes the query, delegates to specialists via the shared global context, and synthesizes their observations into a final, verifiable answer.


menti.com
4279 1722

The checkout API has been slowing down for two days. No deployment occurred, and there are no errors in the logs, but a feature flag controlling a third-party shipping-rate integration was changed three days ago. How should the Triage Agent most likely proceed, and why would a single flat ReAct agent (with all enterprise tools loaded) be riskier here?

A) Codebase Agent first, then Infra; flat agent fine — only two tools needed.

B) Infra Agent + HR/Policy Agent; risk only applies to tool-heavy queries.

C) Infra Agent (latency) + config/flag owner, then synthesize; flat agent with many tools more likely to mis-select/hallucinate params.

D) Codebase Agent only — no deploy rules out config/infra; ReAct can’t use Observations.

Answer: C

- **C is correct** - it matches the actual symptom (config/flag change, not code) and correctly explains that tool-hallucination risk is a property of the agent's *total* tool count in the live system, not of how many tools a given query happens to need.
- **A is wrong** twice over: it misdiagnoses the cause (starts with code despite no deploy), and assumes a flat agent is "fine" just because this query is simple - ignoring that the same agent still carries all 30+ tools in production.
- **B** gets the delegation right but the reasoning wrong: it argues the risk only exists for tool-heavy queries, when the lecture frames tool hallucination as a standing property of the agent's tool count, not a per-query risk.
- **D** is wrong on the facts: ReAct's defining feature is that it *does* use Observations to ground reasoning - this option misstates the framework itself.

Caveats in Production: The Planning Trap

How do we write this in Python?

The Risk of Loops

Using pure `while` loops is dangerous and hard to debug in complex production environments.

State Management

We need a structured way to manage the state of the agent to ensure predictability and traceability.

Note: Enter State Machines.

State Machines & LangGraph

Core Transition

Moving from `Linear code` to `Graphs`.

Industry Impact: This is the standard for building resilient, production-ready agentic systems.

What is a State Machine?

Core Definition
A mathematical model of computation used to design algorithms.

State Integrity
The system is in exactly one of a finite number of states at any given time.

Transitions
Rules that dictate how the system moves from one state to another based on inputs.

Agentic Context: States represent "Planning," "Executing Tool," or "Generating Final Answer."

Why State Machines for AI?

The Challenge: Non-Determinism

AI outputs are inherently non-deterministic, making them difficult to predict and control in complex workflows.

The Solution: Deterministic Pathways

State machines force deterministic pathways on top of LLM outputs, ensuring the system follows a predefined logic.

Visual Analogy: It creates a rigid track for the train (the LLM) to run on, preventing it from derailing.

Introduction to LangGraph

Library Foundation

A library built on top of LangChain to build stateful, multi-actor applications with LLMs.

Graph Modeling

Models the application as a cyclic graph. Unlike standard LangChain which uses DAGs (Directed Acyclic Graphs), LangGraph allows cycles.

Key Advantage: Cycles are essential for the "Think → Act → Observe → Think" loop in agentic systems.

Context: What is LangChain?

The Foundation

An open-source framework designed to simplify the creation of applications powered by LLMs. Provides standard interfaces for Prompts, Models, Tools, and Vector Stores.

The Concept of "Chains" & Limitations

Chains link components into DAGs (Embed → Retrieve → LLM → Answer). However, DAGs only flow in one direction and cannot loop for reasoning.

The Relationship: LangGraph & LangChain

LangGraph is built *on top* of LangChain. LangChain builds the tools (engine parts), while LangGraph wires them into cyclic loops (transmission).

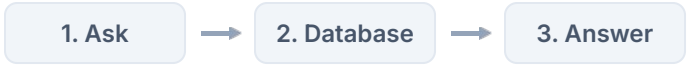
Integration: In our architecture, we use LangChain to build individual tools and LangGraph to wire them into stateful loops.

LangChain vs. LangGraph: The Core Difference

Scenario (Building a Support Assistant): Imagine an AI that answers questions, checks order databases, and offers refunds.

LangChain: Linear Pipeline

Connects components in a straight, step-by-step line (A → B → C). It cannot natively loop backward.

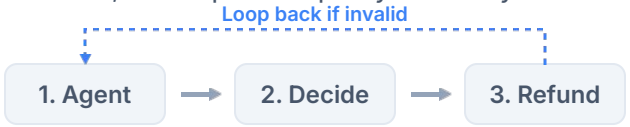


Best For:

Predictable, sequential tasks like summarizing documents or basic Q&A retrieval.

LangGraph: Cyclic Graph

Allows the AI to loop, make decisions, correct its own mistakes, and repeat steps dynamically.



Best For:

Complex AI agents requiring trial-and-error, human-in-the-loop approval, or multi-step reasoning.

Key Takeaway for Speaker:

LangChain is like an *assembly line* (one-way flow), while **LangGraph** is like a *flowchart* (supports loop-back decisions).

Core Concepts of LangGraph: The State

State Definition

A shared data structure (usually a Python `TypedDict`) passed between every node in the graph.

Incremental Updates

Nodes update the state; they do not overwrite it completely.

Analogy: The Shared Meeting Room Table

In our multi-agent team, the State acts as the shared table. The Triage Agent drops a plan, the Infrastructure Agent reads it, performs work, and logs the results. Everyone reads from and writes to the same State.

Core Concepts of LangGraph: Nodes

Nodes: The Workhorses

Python functions (or classes) that receive the State, perform work, and return a state update.

Nodes as Specialized Agents

In our architecture, Nodes represent specialized agents and their tools:

`TriageNode, InfrastructureNode, CodebaseNode`

Note: A node is where actual LLM compute or tool execution happens. We use dedicated nodes for each team member instead of a single "Agent Node".

Core Concepts of LangGraph: Edges

Edges: The Traffic Controllers

Determine which node runs next in the application flow.

Conditional Edges

A routing function that inspects the `State` to dynamically decide the next step based on logic or LLM output.

Note: The "Manager" Role

For our project, the conditional edge acts as the manager. After the Triage Agent analyzes the prompt, the conditional edge asks, *"Who should handle this?"* and routes the flow to the **Infrastructure** or **Codebase** node accordingly.



menti.com

4279 1722

A single ReAct agent has 30 tools and a correctly working long-term memory system. It still inefficiently resolves recurring multi-day incidents. What's structurally missing, and why doesn't more memory or more tools fix it?

A) Bigger context window for tool outputs.

B) Faster model for quicker tool calls.

C) Needs hierarchical agents (Triage + specialists) or Planner/Executor split — past ~20 tools, hallucination rises regardless of memory; memory ≠ action-selection.

D) More frequent memory writes.

Answer: C

- **C is correct** - the lecture sets roughly 20 tools as the threshold past which a single agent suffers tool hallucination (confused schemas, guessed parameters), independent of memory quality. Memory governs *what* the agent knows; tool count and reasoning load govern whether it can reliably *act*. Only a structural split (multi-agent or Planner/Executor) addresses this.
- **A** repeats the "more context is always better" misconception explicitly flagged as false (the needle-in-a-haystack caveat) - and a bigger window doesn't fix parameter/schema confusion anyway.
- **B** misreads "efficiently" as a speed problem; a faster model with the same 30-tool overload would still hallucinate calls, just faster.
- **D** is contradicted by the premise - memory is already stated to be "correctly implemented," so the failure isn't a recall problem.

Designing the Multi-Agent DevOps Graph (Part 1)

State Schema Definition:

`messages`
List of chat messages (the conversation history).

`next_agent`
String (e.g., "infrastructure", "codebase", "FINISH").

`sender`
String (tracks which agent last spoke).

Schema Example (State):

```
# Current state of the DevOps graph
state = {
  "messages": [HumanMessage("Check logs")],
  "next_agent": "infrastructure",
  "sender": "triage"
}
```

Note: Before writing any application logic, we define the schema.

We must track **`next_agent`** to know where to route, and **`sender`** to maintain an audit trail of which specialist provided the data.

Designing the Multi-Agent DevOps Graph (Part 2)

Node 1 (Triage Agent)

Reads user query, determines the required domain, updates `next_agent = "infrastructure"`.

Node 2 (Infrastructure Agent)

Triggered by the router. Sees it needs to check logs. Calls its Splunk tool, gets results, updates `messages` with findings, and sets `next_agent = "triage"`.

Note: The Collaborative Pattern

The specialist agent does its job and immediately hands control back to the Triage Agent, which synthesizes the final answer or plans the next step.

Designing the Multi-Agent DevOps Graph (Part 3)

The Routing Logic (Conditional Edge):

- Read `State["next_agent"]`.
- If `"infrastructure"`, go to `InfrastructureNode`.
- If `"codebase"`, go to `CodebaseNode`.
- If `"FINISH"`, exit graph and display to user.

Note:

This replaces brittle **`if/else`** statements in your code. The LangGraph engine handles the physical traversal between these isolated agent nodes.

Code Abstraction: Wiring the Multi-Agent Team

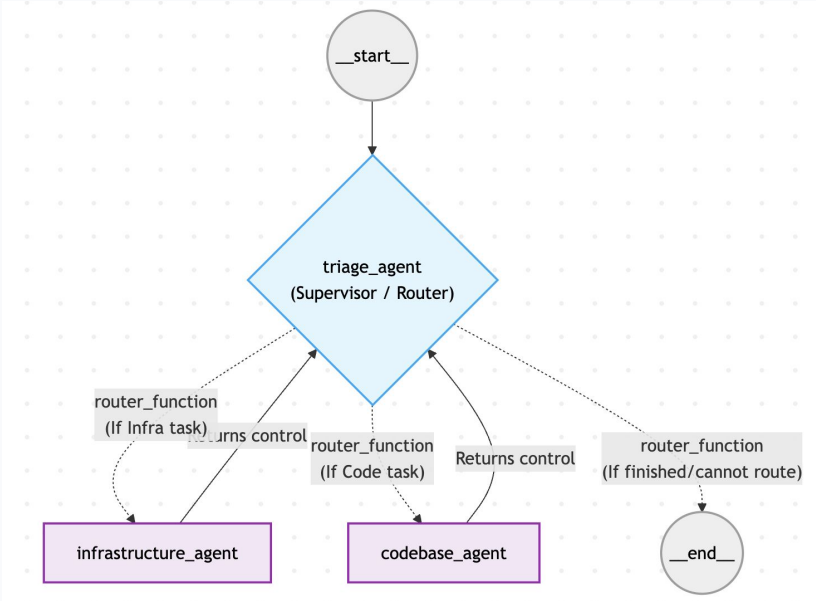
```
graph = StateGraph(AgentState)

graph.add_node("triage_agent", triage_node)
graph.add_node("infrastructure_agent", infra_node)
graph.add_node("codebase_agent", code_node)

# The Triage agent decides who acts next
graph.add_conditional_edges("triage_agent",
router_function)

# Specialists always return control to Triage after
finishing
graph.add_edge("infrastructure_agent",
"triage_agent")
graph.add_edge("codebase_agent", "triage_agent")

app = graph.compile()
```



Note: LangGraph's API is highly declarative. We define the agents, map the transition rules, and compile it into an executable app.

The Power of compile()

Final Executable Application

Compiling the graph transforms your declarative agent definitions into a high-performance, production-ready executable.

Thread Checkpointing

Built-in persistence layers allow the state machine to pause and resume seamlessly across threads.

Execution Streaming

Enables real-time data flow monitoring and incremental updates during complex agent orchestration.

Note:

Compilation is where LangGraph sets up the underlying infrastructure to pause, resume, and track the state machine across our multi-agent network.

Human-in-the-Loop (HITL)

Controlled Execution via State Machines

State machines allow us to pause execution at specific nodes.

Use Case: Safe Tool Deployment

Pausing before a destructive tool is run. The agent must wait for human confirmation before the state transitions.

Mandatory Enterprise Security

You do not want autonomous LLMs silently altering infrastructure. HITL is a critical safety feature to prevent unintended changes.

Project Context: Pausing the Infrastructure Agent

Scenario

Infrastructure Agent detects a memory leak and proposes restarting the Kubernetes Pod via the `kubectl_restart` tool.

Graph Action

Reaches an `approve_action` node and suspends the state.

Human Action

Slack message sent to Admin. Admin clicks "Approve." Graph resumes and restarts the pod.

Note:

Because the entire state is serialized by the orchestrator, we can pause the graph for hours or days while waiting for the human, without losing context.

Caveats in Production: Multi-Agent Infinite Loops

The Danger

The Triage Agent asks the Codebase Agent for a file. The file doesn't exist. The Codebase Agent asks Triage for clarification. Triage asks Codebase again. Infinite loop.

The Fix

Always set a strict **`recursion_limit`** parameter in LangGraph (e.g., max 15 steps) to kill runaway loops.

Note:
Agents arguing with each other in a loop will burn thousands of API tokens per minute. Guardrails are essential.

Caveats in Production: Multi-Agent State Bloat

The Danger: Rapid Context Growth

Every time the Triage Agent and Infrastructure Agent talk, messages are appended to the State. The LLM's context window fills up rapidly, leading to performance degradation.

The Fix: Automated Summarization

Implement a **`Summarize`** node that triggers when **`len(messages) > 10`**. It compresses older inter-agent dialogue into a short, manageable summary.

Key Takeaway

Managing state size is critical for maintaining fast response times and keeping API costs low. Efficiency is a production requirement.

How does the multi-agent team remember?

Beyond Temporary State

While state is the immediate data of a single execution thread, true memory allows agents to persist knowledge across different sessions and share historical context globally.

Contextual Insight:

State is temporary and thread-specific. Let's explore how agents utilize true memory to maintain historical context and shared learning.

The Concept of Agentic Memory

Stateless by Default

LLMs are stateless by default. Every single API call to an agent starts blank.

The Memory Mechanism

Memory is the mechanism of injecting relevant past context into the current prompt.

Critical Impact:

Without memory, the Triage agent would forget the initial user request the moment it receives a payload back from the Infrastructure agent.

Types of Memory

Episodic Memory

(Short-Term)

The current conversation thread within the State.

Semantic Memory

(Long-Term)

Facts learned and stored over time across different sessions.

Mental Model:

Think of **Episodic memory** as the agents' shared **RAM** during a live incident, and **Semantic memory** as the company's shared **Hard Drive**.

Episodic Memory

What it is:

Keeping track of the immediate back-and-forth between the user and the sub-agents.

How it is used:

Passing the list of `Message` objects in the State.

LangGraph Feature: Thread Checkpointing

Note:

Checkpointing saves the State at every node execution to a database (like Postgres or SQLite) so the agents don't lose their train of thought.

Episodic Memory in DevOps Project

Turn 1

User: "Check DB server." → **Triage Agent:** Queries Infra Agent → **Infra:** "It's running at 90% CPU."

Turn 2: The Memory Advantage

User: **"Why is *it* so high?"**

Because of **Episodic Memory** in the State, the Triage Agent:

- Resolves **"it"** to the DB server automatically.
- Immediately routes the query to the **Codebase Agent** to check recent commits, without asking the user for clarification.

Note: This is crucial for natural, multi-turn debugging and maintaining a fluid developer experience.

The Limit of Episodic Memory

Scale & Efficiency Constraints

- Conversations grow indefinitely, leading to context window saturation.
- Excessive token usage causes latency spikes and exponential cost growth.
- Impractical to pass multi-month incident histories into every single agent prompt.

Strategic Pivot Required

This architectural limit defines the point where we must transition from Episodic to **Long-Term Memory**.

Semantic Memory(Long-Term)

What it is:

Persisting facts, user preferences, and historical solutions across different chat sessions.

How it is used:

Using a Vector Database to store summarized learnings, and a Retriever to fetch them.

Autonomous RAG Architecture

Note:

Semantic memory(Long-term) is essentially an autonomous RAG system where the agents write the documents themselves.



menti.com

4279 1722

Leadership wants both mandatory human approval before infrastructure changes *and* automatic self-healing for previously-approved recurring fixes. Which design reconciles both safely?

A) HITL gates only unmatched actions; approved fixes saved with exact scoped condition + timestamp, future matches skip HITL; recursion limit still bounds attempts.

B) HITL on every action, no exceptions; memory only briefs human.

C) Disable HITL after one month.

D) Raise recursion limit so agent retries instead of involving human.

Answer: A

- **A is correct** - it satisfies *both* requirements: every genuinely new action is still gated by a human, while genuinely repeat fixes (the Day-1-to-Day-30 self-healing pattern from the lecture) bypass the gate because a human already approved that exact, scoped condition.
- **B** satisfies the approval requirement but eliminates self-healing entirely - nothing is ever automatic, which fails half the brief.
- **C** removes the safety net altogether, which the leadership's first requirement explicitly rules out.
- **D** confuses the recursion limit's actual role (bounding loops within one resolution attempt) with approval-gating, and doesn't address *reuse* of previously approved fixes at all - it's solving a different problem than the one asked.

Writing to Semantic Memory(Long-Term)

Specific Agent Tool

```
Save_To_Knowledge_Base(fact: str)
```

Synthesis & Storage Process

When the team solves a novel issue (e.g., Codebase agent finds a bug, Infra agent fixes it), the Triage Agent:

- Synthesizes the technical solution into a portable fact.
- Writes the new entry directly to the **Vector DB**.

Autonomous Learning Advantage:
The system gets smarter over time without requiring human engineers to manually write new runbooks.

Reading from Semantic Memory(Long-Term)

Query & Retrieval Mechanism

Before formulating a complex plan, the Triage Agent executes a silent search via the Policy & HR Agent (or a Knowledge tool): *"Have we seen a similar error before?"*

If yes, the past solution is injected into the prompt, bypassing the need to deploy the Codebase or Infra agents.

Key Advantages

This drastically **speeds up resolution time** for recurring bugs and **saves compute costs** by avoiding expensive agent deployments.

The Self-Healing System in Action

Day 1: Novel Issue Discovery

Triage coordinates Infra and Codebase agents to debug a weird DNS issue.

Effort: 15 API Calls

Knowledge Saved: "DNS issue X requires flushing cache Y."

Day 30: Automated Healing

Same issue occurs. Triage checks long-term memory and directly outputs the solution.

Effort: 2 API Calls (86% Reduction)

Action: Solution applied instantly from memory.

The Ultimate Goal: Compounding Institutional Knowledge

By persisting technical findings into long-term memory, the enterprise AI system automatically builds an evolving knowledge base that speeds up resolution times and reduces costs with every incident solved.

Caveats in Production: Memory Contradiction

The Problem

The Triage Agent saves outdated facts (e.g., "Server X is the primary DB"). When infrastructure is upgraded, agents hallucinate based on stale, old memory.

The Fix

Memory must include a **`timestamp`** and an **`Update_Memory`** tool to overwrite stale facts.

Note

Vector DBs don't naturally delete old data. You must build **memory-pruning logic** into your agent's capabilities.

Caveats in Production: The "Needle in a Haystack"

The Challenge

Even with long context models (e.g., 1M tokens), injecting too much historical memory dilutes the agents' attention.

The Solution

Configure the retrieval step to only pull the top 3 most relevant memories.

Key Insight

More context is not always better context.



menti.com

4279 1722

Three weeks in, average incident-resolution time - which had been falling - starts rising again, and API costs spike even though incident volume stays flat. Which pair of causes is most plausible, with a diagnostic step to tell them apart?

A) Vector DB storage full; pricing change. Diagnostic: check invoice.

B) Memory contradiction growing messages inflate tokens. Diagnostic: check failures vs. memory age, and message-length growth over time.

C) Recursion limit too high; Triage deleted. Diagnostic: restart service.

D) More engineers hired; model downgraded. Diagnostic: ask engineers.

Answer: B

- **B is correct** - both causes are named, specific architectural failure modes from the lecture, and each has a falsifiable, targeted diagnostic that distinguishes it from the other.
- **A**'s causes are generic infrastructure/billing issues never tied to this architecture, and "check the invoice" only confirms a symptom (cost), not a cause.
- **C** doesn't fit the timeline: a too-high recursion limit would cause near-immediate runaway cost, not a delayed, gradual rise; "Triage deleted" would cause total failure, not degradation; "restart the service" isn't diagnostic.
- **D** directly contradicts the premise (incident volume is stated as flat) and proposes causes that aren't verifiable from the system's own state.

Provenance and Citations

Building Trust in Autonomous Multi-Agent Systems

Establishing a verifiable chain of custody for every decision and discovery made within the swarm.

The Accountability Requirement

Scenario: If the Triage agent attributes an incident to a specific code commit, you must identify exactly which sub-agent made the discovery and the specific location of the evidence.

What is Provenance?

General Definition

The origin or source of a piece of information.

AI Provenance

Mapping every claim in a generated answer back to specific components:

- Specific documents or log lines
- Tool outputs or sub-agent actions

How It Is Achieved

Provenance is achieved by **Citations**, establishing a verifiable chain of custody for every autonomous decision and discovery.

Note: Without provenance, a multi-agent system is just a massive black box generating text.

Why Citations Matter

Verification

Human engineers must be able to audit the AI's logic.

Hallucination Mitigation

Forcing agents to cite sources drastically reduces fabricated answers.

Accountability

Knowing if the error came from the Codebase Agent's analysis or the Infra Agent's log pull.

Note

In enterprise support, an **uncited answer** is an **unusable answer**.

The Spectrum of Citation Architectures

Generating reliable citations is not a solved problem in AI. There are four primary architectural approaches, ranging from easy-but-flawed to complex-but-robust:

1. Inline Prompting
(Zero-Shot / Few-Shot)

2. Structured Tool Forcing
(JSON Schema)

3. Post-Hoc Attribution
(Retroactive Verification)

4. Agentic State Provenance
(LangGraph tracing)

Note: We will evaluate each method based on latency, implementation cost, and susceptibility to "citation hallucination" (where the model cites a document that doesn't exist or doesn't support the claim).

Method 1: Inline Prompting (The Baseline)

Mechanism

You instruct the LLM in the system prompt to append bracketed citations directly into the text stream.

Prompt Example:

"Use the provided context. After every sentence, cite the source like this: [Doc_1]."

Pros

- Very low latency (streams naturally)
- Easiest to implement

Cons

- Highly prone to hallucination
- Relies entirely on instruction-following

Note: Early RAG systems relied on this, but performance degrades quickly as context windows fill up.

Method 2: Structured Tool Forcing (The Enterprise Standard)

Mechanism

Force the LLM to output a strict JSON schema where the answer and the citations are separated into distinct data fields.

Schema Example (JSON):

```
{
"answer": "The server failed due to an OOM error.",
"citations": [ {"doc_id": "splunk_log_42", "exact_quote": "..."} ]
}
```

Pros

Enables programmatic verification of exact quotes in source text before display.

Cons

Cannot be easily streamed; higher latency due to waiting for full JSON object.

Note: Forcing **`exact_quote`** creates a deterministic bridge; if a match isn't found in source, the UI can flag it as invalid.

Method 3: Post-Hoc Attribution (The Fact-Checker)

Mechanism

Decouple generation from citation: A fast LLM generates the answer, then a secondary NLI model or smaller LLM injects citations retroactively by comparing against source docs.

Pros

- Highest accuracy.
- Generator model isn't distracted by citation formatting rules.

Cons

- Doubles the compute cost.
- Heavily increases latency.

Note:

Academic approach gaining traction in high-stakes environments (legal/medical). One model creates fluently, while a strict logical model fact-checks and cites.

Method 4: Agentic State Provenance (The LangGraph Approach)

Mechanism

Citations are not generated by the LLM's text output, but are derived from the execution graph itself.

How it works: Triage Agent delegates to Codebase Agent.

```
> Agent calls GitHub_Tool & gets Commit_88X.
> State records: Message(content="Commit_88X...", name="Codebase_Agent")
```

Pros

100% deterministic. The citation is a literal log of the API call that occurred in the state machine.

Cons

Only works for discrete tool calls, not for synthesizing large paragraphs of retrieved text.

Note: Most secure method for DevOps. The UI builds a visual audit trail from LangGraph State history, bypassing LLM text uncertainty.

Citation Architectures: Trade-off Matrix

Method	Implementation	Latency	Accuracy / Reliability
Inline Prompting	Low (Text only)	**Very Fast**	Low (Prone to hallucination)
Structured Output	Medium (JSON parsing)	**Moderate**	High (Exact quote verification)
Post-Hoc Attribution	High (Multi-model pipeline)	**Slow**	Very High (Strict entailment)
Agentic State	High (LangGraph setup)	**Graph-dependent**	**Absolute (Deterministic tool logs)**

Note: For a consumer chatbot, Inline Prompting is often enough. But for our Enterprise DevOps agent, we must use a combination of Structured Outputs for document RAG, and Agentic State for API tool executions.

Project Context: Multi-Agent Citations

Final Output to User

"The payment container failed due to an Out of Memory error **[Infra Agent: Splunk-pod-1, Line 452]**. This aligns with yesterday's commit increasing the cache size **[Codebase Agent: Git Commit 8f4b2a]**."

Note:
This gives the human engineer immediate confidence and direct links to verify the work of both specialist agents.



menti.com

4279 1722

A team uses only Inline Prompting for citations in its enterprise DevOps assistant, reasoning "the model is reliable enough." What two risks does this leave open, and what's a minimal fix (short of full Post-Hoc Attribution)?

A) Tool hallucination; state bloat. Fix: add Summarize node.

B) Citation hallucination stale memory cited as current. Fix: verify cited IDs vs. message history + timestamp/prune memory.

C) Context token limits; recursion limit. Fix: raise both.

D) Citations can’t be hyperlinked; too slow. Fix: switch fully to Post-Hoc.

Answer: B

- **B is correct** - these are the two specific failure modes tied to *this exact choice*: Inline Prompting has no built-in safeguard against fabricated citations, and uncurated long-term memory can surface outdated facts as if current. The fix mirrors the lecture's actual countermeasures (state-verification + timestamp/prune), getting most of the safety of Post-Hoc Attribution without its full cost.
- **A**'s risks are real elsewhere in the architecture but are caused by tool count and message growth, not by the citation method - a non sequitur relative to the question.
- **C** conflates an unrelated capacity constraint (context window, recursion limit) with a citation-architecture decision.
- **D** contains a factual error (inline citations are fast, not slow) and proposes a non-minimal fix the question explicitly asked to avoid.

The UI/UX of Citations

Don't just show [1].

Hyperlink to Enterprise Systems

Link directly to sources like live Splunk dashboards or GitHub PRs to enable immediate verification.

Note:

User experience drives adoption of AI tools. Make verification frictionless to build trust.

Post-Processing: Verifying Citations

The Problem: Hallucination

LLMs sometimes hallucinate citations (e.g., citing a commit hash that wasn't provided in the Codebase Agent's payload).

The Solution: State-Verification Step

Add a lightweight verification step in code that ensures every cited ID actually exists in the State's message history.

Core Principle:
Never trust the LLM blindly. Always verify the citation format programmatically before showing it to the user.

Connecting Provenance to State Machines

Native Provenance

LangGraph makes provenance natively easier by leveraging the persistent state of graph execution.

Chronological Audit Trail

Because every sub-agent execution and tool call is saved in the State, you have a perfect chronological audit trail of exactly what the team did.

Visualizing the "Thought Process"

You can render the LangGraph execution path in your UI to show the user the multi-agent reasoning journey visually.

Summary of Multi-Agent Architecture

Router/Triage
Delegates the query to the appropriate agent based on task requirements.

Specialist Agents
Execute narrow, domain-specific tasks such as Codebase analysis or Infra management.

Orchestrator (LangGraph)
Manages shared state, control flow edges, and iterative loops across the system.

Memory
Provides context through Episodic Threading and Long-Term Vector databases.

Blueprint for Scale:
This architecture serves as the foundational blueprint for a production-grade enterprise agent network.

Summary: The DevOps Agent Journey

From basic search (RAG) to an **autonomous, stateful team of specialist agents** that can:

- Read logs and check code
- Remember past fixes
- Provide perfectly cited, multi-hop incident summaries

Conclusion

You now have the theoretical foundation to build this system.

Project Implementation Steps

01 Define your shared **Agent State schema**.

02 Build your **specialist tools** (Python functions for Splunk/GitHub).

03 Write your **nodes** (`triage`, `infra`, `codebase`).

04 Wire the **routing edges** and compile.

Note:
Start simple. Build the Triage and Infrastructure agent flow first before adding the Codebase agent.

Best Practices: Inter-Agent Error Handling

Defensive Tool Execution

Wrap tool executions in `try/except` blocks to catch and manage runtime failures.

Fidelity of Feedback

If the Codebase Agent fails an API call, return the exact error string back to the State for inspection.

Autonomous Remediation

Allow the agent to read errors and fix its parameters before returning to Triage.

Note on Self-Correction:
Agents are remarkably good at self-correcting if you provide the full stack trace.

Best Practices: System Prompts

Core Principle
Give each agent a strict persona, goal, and boundary rules.

Triage Prompt
"You are a routing manager. Do not answer technical questions directly. Delegate to specialists."

Infra Prompt
"You are a DevOps engineer. Query logs. NEVER suggest deleting a database."

Key Note:
Specialized system prompts are the primary guardrails for multi-agent behavior.

Multi-Agent Orchestration Strategy: Key Takeaway 1

Core Architecture: Separation of Concerns

Multi-agent orchestration separates **planning (Triage)** from **execution (Specialists)**.

Key benefits include:

- Reduced context exhaustion
- Minimization of hallucinated tool calls

Strategic Advantage

Specialization breeds reliability in complex autonomous systems.

Multi-Agent Orchestration Strategy: Key Takeaway 2

State Machines: The Foundation of Collaboration

State machines (like LangGraph) provide the essential infrastructure for safe agent collaboration:

- Shared memory space for consistent context
- Robust routing logic for task delegation
- Human-in-the-loop checkpoints for safety

The Management Layer

The graph serves as the manager of the team, orchestrating interactions and ensuring reliability.

Multi-Agent Orchestration Strategy: Key Takeaway 3

Memory Management: Episodic vs. Long-term

Memory must be managed carefully to ensure agent efficiency and institutional growth:

- **Episodic Memory:** Handles the immediate multi-agent chatter and transient interaction history.
- **Long-term Memory:** Builds durable institutional knowledge and persistent records.

Critical Safeguard

Ensure agents don't get lost in infinite loops of episodic chatter, which can degrade performance and inflate costs.

Multi-Agent Orchestration Strategy: Key Takeaway 4

Data Integrity: Provenance & Citations

Provenance and citations are non-negotiable for enterprise-grade AI systems:

- Map every claim back to the specific specialist agent.
- Identify the exact enterprise tool that sourced the raw data.

Enterprise Value

Transparency builds enterprise trust, ensuring data accountability across the network.

Conclusion & Next Steps



You are now ready to build stateful, Agentic RAG Systems capable of autonomous orchestration.



Module Complete

Apply these orchestration strategies to your next autonomous project.

Meet the Team

Teaching Assistants (IITKGP)



Instructor

Pawan Goyal



Manoj Balaji Jagadeeshan



Pretam Ray



Kunal Kingkar

Thank You!